Systemadministration – Virtualisering och molntjänster

Anycast Routing

Laboration 5

Poäng och betygsättning

Laborationen betygsätts U/G utan poängsättning. Deadline är en vecka från labbstart.

Utförande

Uppgiften utförs i grupper om två till tre studenter. Tidigare serverkitt delas mellan grupperna. Studenterna ansvarar för gruppindelningen.

Misstänkt fusk, plagiat och trakasserier anmäls till *Disciplin- och avskiljandenämnden* (DAN).

Syfte

Studenten skall visa färdighet i planering och uppsättning av virtualiseringslösningar för servrar, nätverk och applikationer. Studenten skall visa förmåga att hantera centraliserade lagringslösningar och drift av mindre datacenter. Studenten skall visa förmåga att utforma IT-system som bättre utnyttjar befintliga resurser

Handledning

Handledning sker under ordinarie laborationstillfällen.

Redovisning

Laborationen redovisas i labbsal för närvarande labbhandledare.

1 Introduktion

I denna laboration ska HAproxy-servrarna som använts i tidigare laborationer ersättas. Istället ska nätverket hantera last-balansering med routing protokollet OSPF. Varje sida kommer få en adress, och i och med att samma containrar finns på två enheter kommer dessa adresser inte vara unik (se tabell 1). Dessa adresser kommer med andra ord användas för anycast. Vilket innebär att klienter kommer skickas till närmaste enhet med anycast adressen med hjälp utav routing protokollet.

Container	Adress
sida1	3.3.3.3/32
sida2	4.4.4.4/32

Tabell 1: *Anycast adresser.*

Tips!

Båda docker-maskinerna behöver samma konfiguration. Konfigurera därför de båda parallellt, och dela upp arbetet inom gruppen, för att spara lite tid.

2 Loopback Interface

Logga in på dina Docker-maskiner och kontrollera att dina webb-containrar fortfarande fungerar. Lägg sedan till två nya loopback-adresser enligt figur 1 nedan.

```
source /etc/network/interfaces.d/*

# loopback network interface
auto lo
iface lo inet loopback

# anycast 1
auto lo:0
iface lo:0 inet static
  address 3.3.3.3/32

# anycast 2
auto lo:1
iface lo:1 inet static
  address 4.4.4.4/32

# The primary network interface
allow-hotplug eth0
iface eth0 inet static
        address 10.0.40.100/24
        gateway 10.0.40.1
```

Figur 1: *Nätverkskonfiguration med två anycast adresser.*

Applicera nätverkskonfigurationen med utdrag 1 och kontrollera att de adresserna dyker upp.

```
sudo ifup lo:0
sudo ifup lo:1
```

Utdrag 1: *Applicera nätverksändringar i Debian.*

Justera sedan containrarna så att de använder de nya loopback-adresserna (se utdrag 2).

```
# stop previous containers
docker container ls
docker conatiner stop sida1
docker container stop sida2
# remove previous containers
docker container ls -a
docker container prune
# start new containers attached to specific ip addresses
docker container run -d --name sida1 -p 3.3.3.3:80:80 sida1:1.0
docker container run -d --name sida2 -p 4.4.4.4:80:80 sida2:1.0
docker container ls
```

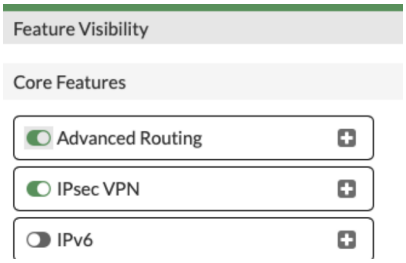
Utdrag 2: *Starta nya containrar kopplade till olika anycast adresser.*

Kontrollera att du kan nå sida1 respektive sida2 från de båda loopback adresserna med **curl** ifrån dina Docker-maskiner.

3 OSPF

3.1 OSPF: FortiGate

Aktivera stöd för OSPF under *"Feature Visibility"* i din VDOM (se figur 2). Glöm inte att klicka på *"Apply"* längst ner innan du går vidare.

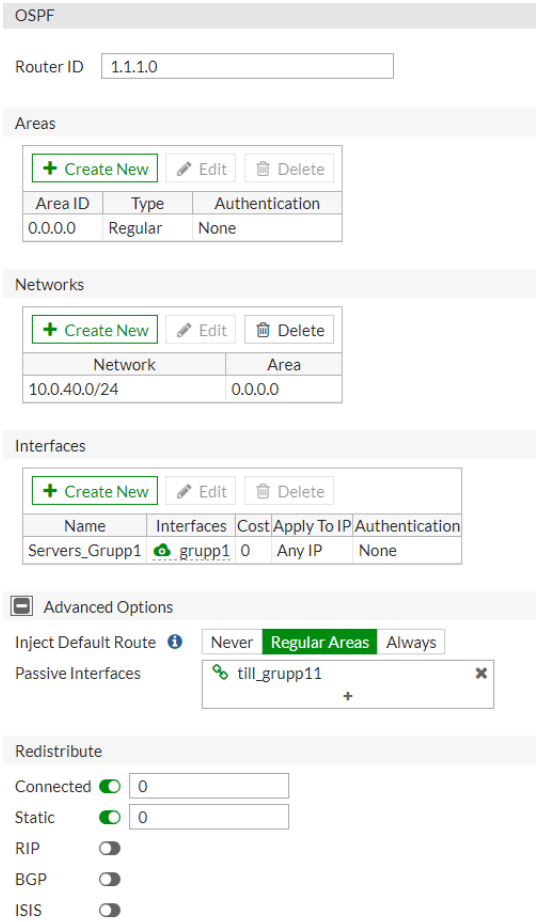


Figur 2: *FortiGate aktivera stöd OSPF med "Advanced Routing".*

Konfigurera OSPF och se till att:

- Ställ in ett unikt router-id för din VDOM
- Koppla din VDOM till area 0 (0.0.0.0)
- Lägg till nätverket 10.0.40.0/24 i area 0
- Lägg till ditt VLAN-interface i OSPF-processen
- Se till ditt interface mot root-VDOM inte skickar eller lyssnar på OSPF-paket
- Se till att default routes sprids ifrån din VDOM med OSPF
- Redistrubera även connected- och static routes

Använd figur 3 som referens.

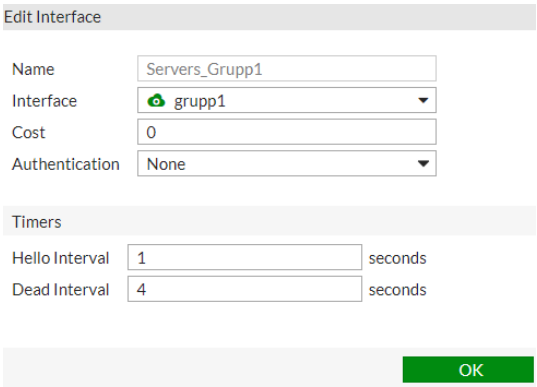


Figur 3: *OSPF konfiguration i FortiGate.*

Se även till att ditt VLAN-interface har:

- Hello interval satt till 1 sekund
- Dead interval satt till 4 sekunder

Använd figur 4 som referens.



Figur 4: *Timer konfiguration i FortiGate.*

Vi behöver även justera prioriteten för ditt VLAN-interface. Dock kan detta inte göras i det grafiska gränssnittet och behöver istället göras ifrån ett CLI. Öppna en konsol genom att klicka på *"CLI Console"* uppe i högra hörnet medans du är inne på din

VDOM. Du bör se din VDOM i prompten i konsol-fönstret. Kontrollera att OSPF-konfigurationen har applicerats. Justera sedan prioriteten för ditt VLAN-interface till 255. Använd utdrag 3 nedan som guide.

```
config router ospf
    # check ospf configuration
    show

    # configure ospf priority
    config ospf-interface
        edit "Servers_Grupp1"
        set priority 255
    next
    end

    # check new ospf configuration
    show
end
```

Utdrag 3: *OSPF priority konfiguration.*

3.2 OSPF: Linux

För att Linux ska kunna dela routes med din VDOM behöver vi en OSPF-mjukvara på den. Därför kommer vi installera *Free Range Routing* (FRR) för att få tillgång till routing-funktioner i Linux (se utdrag 4).

```
sudo apt update
sudo apt install frr frr-pythontools
```

Utdrag 4: *Installation av frr.*

Se till att OSPF är aktiverat i **/etc/frr/daemons**. Det vill säga att **ospfd=yes** är satt.

Konfigurera sedan FRR i filen **/etc/frr/frr.conf**. Använd utdrag 5 som referens. Observera dock att dina interface kan ha andra namn. Notera även att **ospf router-id** måste vara unikt.

```
!
hostname webserver1
log syslog informational
log file /var/log/frr/frr.log
!
interface lo
  ip ospf cost 500
!
interface eth0
  ip ospf cost 500
  ip ospf hello-interval 1
  ip ospf dead-interval 4
!
router ospf
  ospf router-id 1.1.1.1
  log-adjacency-changes detail
  default-metric 10
  network 10.0.40.0/24 area 0.0.0.0
  ospf opaque-lsa
!
line vty
!
```

Utdrag 5: *Konfiguration av frr.*

Applicera konfigurationen med **sudo systemctl restart frr**.

Förutsatt att det inte finns några felmeddelanden i FRRs loggar så kan du ansluta till din *Free Range Router* (se utdrag 6). Du kan ignorera eventuella varningar om **"zebra: pid file not found"**. Använd sedan **show**-kommandon för att kontrollera att OSPF fungerar. Kontrollera åtminstone att alla neighbors hittas, vilka routes som delas och vilka routes som har installerats.

```
sudo vtysh
```

Utdrag 6: *Anslut till FRR.*

4 Health Checks

Syftet med laborationen är att utnyttja anycast för att lastbalansera webbtrafik och adresserna bör därför inte delas om webb-containrarna inte fungerar. Det är därför inte tillräckligt att enbart lägga till nätverken i OSPF-processen på vardera Docker-host. För att lösa detta krav behöver vi implemntera *health-checking* likt *HAproxy*.

På Canvas finns två Bash-skript för att kontrollera om sida1- och sida2-containrarna svarar på HTTP GET-frågor. Ladda upp skripten på dina Docker-hosts och testkör dom manuellt. Observera att du antaligen behöver ändra domännamnen i skripten.

Du bör se att anycast adresserna har lagts till i och deltas med OSPF efter dina tester. Kontrollera båda med **vtysh** och FortiGates konsol. Använd figur 5 som referens.

```
Labbkitt1 # config vdom

Labbkitt1 (vdom) # edit Grupp1
current vf=Grupp1:2

Labbkitt1 (Grupp1) # get router info ospf neighbor

OSPF process 0, VRF 0:
Neighbor ID     Pri   State           Dead Time   Address         Interface
1.1.1.1           1   Full/DR         00:00:04    10.0.40.100     grupp1


Labbkitt1 (Grupp1) # get router info ospf route

OSPF process 0:
Codes: C - connected, D - Discard, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2

O  3.3.3.3/32 [510] via 10.0.40.100, grupp1, Area 0.0.0.0
C  10.0.40.0/24 [10] is directly connected, grupp1, Area 0.0.0.0


Labbkitt1 (Grupp1) #
```

Figur 5: *Kontroll av OSPF i FortiGate.*

4.1 Placering av health checks

Placera skripten under **/usr/local/sbin/** och ställa in ägandeskap och rättigheter enligt:

- användägare: **root**
- gruppägare: **root**
- rättigheter: **754**

4.2 Schemaläggning av health checks

Skripten behöver också schemaläggas så att anycast-adresserna tas bort om en container skulle sluta svara. Skapa en ny cron-fil och se till att skripten körs varje minut utav användaren root (se utdrag 7).

```
cat /etc/cron.d/health-checks
# /etc/cron.d/health-checks

SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

* * * * * root  systemd-cat -t 'health-check-sida1' /usr/local/sbin/
    health-check1.sh
* * * * * root  systemd-cat -t 'health-check-sida2' /usr/local/sbin/
    health-check2.sh
```

Utdrag 7: *Ny cron-fil.*

Se även till att ställa in ägandeskap och rättigheter enligt:

- användägare: **root**
- gruppägare: **root**
- rättigheter: **644**

5 Redovisning

Kontrollera om lastbalanseringen fungerar genom att se på containrarnas loggar medans du skickar webbförfrågningar ifrån en klient (se utdrag 8).

```
docker logs -f sida1
docker logs -f sida2
```

Utdrag 8: *Container loggar.*

Kontrollera om dina *health checks* fungerar genom att stänga av en container. Du bör kunna se loggar ifrån ditt cron-jobb med **sudo journalctl -fu cron**.

? Fråga 1

Redovisa i labbsal att ovan kontroller fungerar.